

Sorting Algorithms

Lesson Objectives

By the end of this lesson, students should be able to:

- Understand the concept and purpose of sorting.
 - Write pseudocode for common sorting algorithms (Bubble Sort, Insertion Sort, Selection Sort).
 - Compare the efficiency of sorting algorithms using time complexity.
 - Apply sorting to solve practical problems such as ranking student marks.
-

1 Introduction to Sorting

Sorting means arranging data in a specific order — usually **ascending** or **descending**.

For example:

Unsorted: [45, 12, 78, 34]

Sorted (ascending): [12, 34, 45, 78]

Sorting makes data easier to search, analyze, and visualize.

2 Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

◆ Pseudocode

```
PROCEDURE BubbleSort(A, N)
  FOR i ← 1 TO N-1
    FOR j ← 1 TO N-i
      IF A[j] > A[j+1] THEN
        TEMP ← A[j]
```

```

        A[j] ← A[j+1]
        A[j+1] ← TEMP
    ENDIF
ENDFOR
ENDFOR
ENDPROCEDURE

```

◆ Example Dry-Run

For A = [5, 3, 8, 4]:

Pass 1: [3, 5, 4, 8]

Pass 2: [3, 4, 5, 8]

Pass 3: [3, 4, 5, 8] → Sorted!

🧠 **Time Complexity: $O(n^2)$**

3 Insertion Sort

Insertion Sort builds the final sorted array one element at a time.

◆ Pseudocode

```

PROCEDURE InsertionSort(A, N)
    FOR i ← 2 TO N
        key ← A[i]
        j ← i - 1
        WHILE j > 0 AND A[j] > key
            A[j+1] ← A[j]
            j ← j - 1
        ENDWHILE
        A[j+1] ← key
    ENDFOR
ENDPROCEDURE

```

◆ Example

For A = [5, 2, 4, 6]:

Step 1: [2, 5, 4, 6]

Step 2: [2, 4, 5, 6] → Sorted!

 **Time Complexity: $O(n^2)$**

Selection Sort

Selection Sort repeatedly selects the smallest element and moves it to the correct position.

◆ Pseudocode

```
PROCEDURE SelectionSort(A, N)
  FOR i ← 1 TO N-1
    min_index ← i
    FOR j ← i+1 TO N
      IF A[j] < A[min_index] THEN
        min_index ← j
      ENDIF
    ENDFOR
    SWAP A[i] WITH A[min_index]
  ENDFOR
ENDPROCEDURE
```



◆ Example

For A = [29, 10, 14, 37, 13]:

Step 1: [10, 29, 14, 37, 13]

Step 2: [10, 13, 14, 37, 29]

Step 3: [10, 13, 14, 37, 29]

Step 4: [10, 13, 14, 29, 37] → Sorted!

 **Time Complexity: $O(n^2)$**

5 Comparing Sorting Algorithms

Algorithm	Best Case	Worst Case	Type
Bubble Sort	$O(n)$	$O(n^2)$	Comparison-based
Insertion Sort	$O(n)$	$O(n^2)$	Adaptive
Selection Sort	$O(n^2)$	$O(n^2)$	Non-adaptive

6 Activity: Sort Student Marks

Task:

Write pseudocode to sort 10 student marks in **ascending order** using **Bubble Sort**.

Example Input:

[70, 85, 60, 90, 50]

Expected Output:

[50, 60, 70, 85, 90]



7 Mini Project: Sorting Algorithms Visualized

Goal:

Work in groups to create a visual demonstration (using PowerPoint, Python, or Scratch) that shows:

- Step-by-step sorting using at least two algorithms.
- Comparison of efficiency for large vs small data.

Project Deliverables:

- Flowchart or diagram of each algorithm.

- Pseudocode and short explanation.
 - Documentation (2–3 pages) describing:
 - The algorithm chosen.
 - Group roles and process.
 - Observations on speed and efficiency.
-

